

Day 4 – Arrays and Strings in Java

1D/2D Arrays, Matrix Operations, Strings & StringBuilder

1. Arrays: 1D and 2D

1D Array: An array is a collection of multiple values of the same data type stored under a single variable name.

2D Array: A 2D array stores elements in rows and columns like a table, and is also known as a **matrix**.

Program 1: 1D Array – Read, Sort & Display

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];

        for (int i = 0; i < 5; i++) {
            arr[i] = sc.nextInt();
        }

        Arrays.sort(arr);

        for (int i = 0; i < 5; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Program 2: 2D Array – Read & Display

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] mat = new int[2][3];

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                mat[i][j] = sc.nextInt();
            }
        }

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(mat[i][j] + " ");
            }
        }
    }
}
```

```

        System.out.println();
    }
}

```

2. Max and Min of a 2D Array

Program 3: Find Maximum and Minimum

```

class Main {
    public static void main(String[] args) {
        int[][] matrix = {
            {10, 40, 20},
            {45, 6, 35}
        };

        int min = Integer.MAX_VALUE;
        int max = Integer.MIN_VALUE;

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                if (min > matrix[i][j]) {
                    min = matrix[i][j];
                }
                if (max < matrix[i][j]) {
                    max = matrix[i][j];
                }
            }
        }

        System.out.println("minimum: " + min);
        System.out.println("maximum: " + max);
    }
}

```

3. Finding the Number of Rows and Columns

Program 4: Rows & Columns of a Matrix

```

class Main {
    public static void main(String[] args) {
        int[][] mat = {
            {10, 20, 30},
            {40, 50, 60}
        };

        int row = mat.length;
        int col = mat[0].length;

        System.out.println(row);
        System.out.println(col);
    }
}

```

Output:

2
3

4. Diagonal Sum of a Matrix

Program 5: Sum of the Main Diagonal

Main diagonal elements: `mat[0][0]`, `mat[1][1]`, `mat[2][2]`

```
class Main {
    public static void main(String[] args) {
        int[][] mat = {
            {10, 20, 30},
            {4, 7, 12},
            {12, 18, 15}
        };

        int n = mat.length;
        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum += mat[i][i];
        }

        System.out.print("Sum of main diagonal: " + sum);
    }
}
```

Output:

Sum of main diagonal: 32
(10 + 7 + 15 = 32)

Program 6: Sum of the Anti-Diagonal (Left Diagonal)

Anti-diagonal elements: `mat[0][2]`, `mat[1][1]`, `mat[2][0]` → index pairs (0,2), (1,1), (2,0).

```
class Main {
    public static void main(String[] args) {
        int[][] mat = {
            {10, 20, 30},
            {14, 25, 12},
            {12, 18, 15}
        };

        int n = mat.length;
        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum += mat[i][n - 1 - i];
        }

        System.out.print("Sum of anti-diagonal: " + sum);
    }
}
```

```
}  
}
```

Output:

Sum of anti-diagonal: 67
(30 + 25 + 12 = 67)

5. Strings in Java

A **String** in Java is an object that represents a sequence of characters. Strings are widely used to store and manipulate text.

A String in Java is **immutable** – once created, its value cannot be changed.

Program 7: Common String Methods

```
class Main {  
    public static void main(String[] args) {  
        // index: 0123456789...  
        String str = "Hello World";  
        String str1 = "Hello World";  
  
        System.out.println(str.equals(str1));           // compare two strings  
        System.out.println("Length: " + str.length());  
        System.out.println("CharAt(10): " + str.charAt(10));  
        System.out.println("Substring: " + str.substring(6, 11));  
        System.out.println("Upper: " + str.toUpperCase());  
        System.out.println("Lower: " + str.toLowerCase());  
        System.out.println("Contains: " + str.contains("Hello"));  
        System.out.println("Trim: " + str.trim());  
    }  
}
```

6. StringBuilder in Java

A String is immutable, so a **StringBuilder** is used when we need to modify a string frequently without creating new objects – it represents a mutable sequence of characters.

Program 8: StringBuilder Operations

```
class Main {  
    public static void main(String[] args) {  
        String str = "Hello ";  
        String str1 = "World";  
        System.out.println(str + str1);           // concatenation  
  
        StringBuilder sb = new StringBuilder();  
        sb.append("Kolar gold field");  
        System.out.println(sb);  
  
        sb.insert(5, "KGF");                       // specific insert position  
        System.out.println(sb);  
    }  
}
```

```

        StringBuilder sb1 = new StringBuilder("mysore");
        sb1.append("Place");           // added at the back
        System.out.println(sb1);

        sb.delete(6, 9);
        System.out.println(sb);

        System.out.println(sb.toString());           // convert StringBuilder to String
    }
}

```

7. Practice Problem: Reverse a Given String

Program 9: Reverse a String

```

class Main {
    public static void main(String[] args) {
        String str = "Hello World";

        String reversed = new StringBuilder(str).reverse().toString();

        System.out.println("Reversed string: " + reversed);
    }
}

```

Output:

Reversed string: dlrow olleH